



Curso Javascript básico

Aula 07 - Manipulando HTML com JavaScript



Manipulando HTML com JavaScript

DOM – Document Object Model

Quando uma página web é carregada, o navegador cria um DOM – Document Object Model, o DOM é um Modelo de Objeto de Documento que permite o acesso e manipulação ao HTML, ou seja, é a representação de dados dos objetos que compõem a estrutura e o conteúdo de um documento na Web. Assim JavaScript pode acessar e alterar todos os elementos de um documento HTML.



Manipulando HTML com JavaScript

O que é o DOM?

O Document Object Model (DOM) é uma interface de programação para os documentos HTML e XML. Representa a página de forma que os programas possam alterar a estrutura do documento, alterar o estilo e conteúdo. O DOM representa o documento com nós e objetos, dessa forma, as linguagens de programação podem se conectar à página.

Uma página da Web é um documento. Este documento pode ser exibido na janela do navegador ou como a fonte HTML. Mas é o mesmo documento nos dois casos. O DOM (Document Object Model) representa o mesmo documento para que possa ser manipulado. O DOM é uma representação orientada a objetos da página da web, que pode ser modificada com uma linguagem de script como JavaScript.

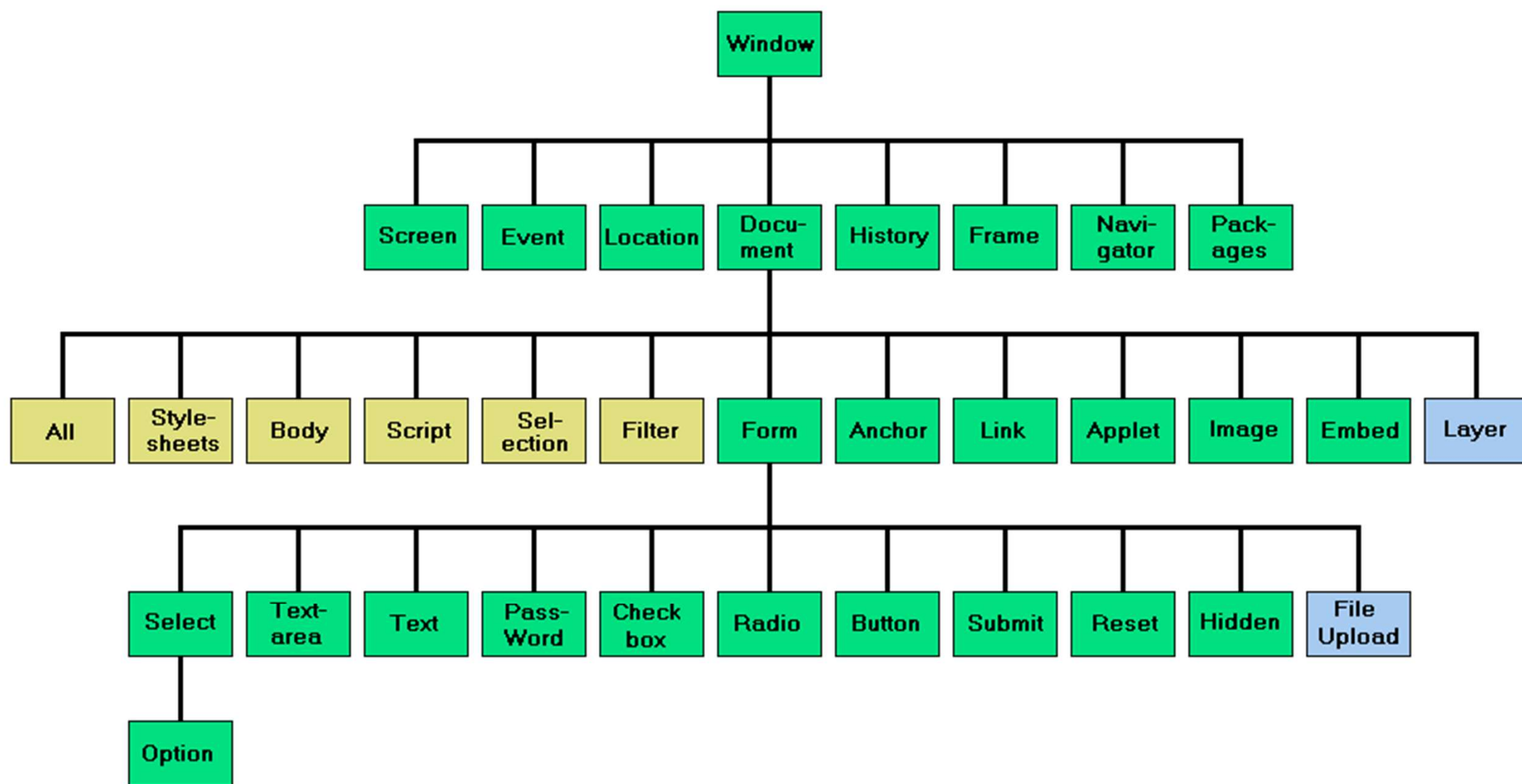


Manipulando HTML com JavaScript

O que é o DOM?

O DOM não é uma linguagem de programação, mas sem ela, a linguagem JavaScript não teria nenhum modelo ou noção de páginas da web, documentos HTML, documentos XML e suas partes componentes (por exemplo, elementos). Cada elemento de um documento - o documento como um todo, o cabeçalho, as tabelas do documento, os cabeçalhos da tabela, o texto nas células da tabela - faz parte do modelo de objeto do documento desse documento, para que todos possam ser acessados e manipulados usando o método DOM e uma linguagem de script como JavaScript.

O DOM foi projetado para ser independente de qualquer linguagem de programação específica, disponibilizando a representação estrutural do documento a partir de uma única API consistente. Embora nos concentremos exclusivamente no JavaScript nesta documentação de referência, as implementações do DOM podem ser construídas para qualquer idioma.





Manipulando HTML com JavaScript

Tipos de dados fundamentais

Document: Quando um membro retorna um objeto do tipo document (por exemplo, a propriedade ownerDocument de um elemento retorna o document ao qual ele pertence), esse objeto é o próprio objeto de document raiz.

Node: Todo objeto localizado em um documento é um nó de algum tipo. Em um documento HTML, um objeto pode ser um nó de elemento, mas também um nó de texto ou atributo.

Element: O tipo do element é baseado em node. Isso se refere a um elemento ou um nó do tipo element retornado por um membro do DOM API. Ao invés de dizer, por exemplo, que o método document.createElement() retorna um objeto de referência para um nó, nós apenas dizemos que esse método retorna o element que acabou de ser criado no DOM. Os objetos do element implementam a interface DOM Element e também a mais básica interface Node, sendo ambas incluídas juntas nessa referência. Em um documento HTML, elementos são ainda mais aprimorados pelas APIs HTML DOM. A interface HTMLElement bem como outras interfaces descrevem capacidades de tipos específicos de elementos (por exemplo, HTMLTableElement para elementos <table>).



Manipulando HTML com JavaScript

Tipos de dados fundamentais

NodeList: Uma nodeList é um array de elementos como os que são retornados pelo método `document.getElementsByTagName()`. Itens numa nodeList são acessados por índices em uma das duas formas:

- `list.item(1)`
- `list[1]`

Esses dois são equivalentes. No primeiro, **item()** é o método único no objeto da nodeList. O último usa uma sintaxe típica de array para buscar o segundo item na lista.

Attribute: Quando um attribute é retornado por um membro (por exemplo, pelo método `createAttribute()`), é um objeto de referência que expõe uma interface especial (embora pequena) para atributos. Atributos são nós no DOM bem como elementos, mesmo que raramente você possa usá-los como tal.



Manipulando HTML com JavaScript

Tipos de dados fundamentais

NamedNodeMap: é como um array, mas os itens são acessados por nome ou índice, embora este último caso seja meramente uma conveniência para enumeração, já que eles não estão em uma ordem específica na lista. Um `namedNodeMap` possui um método `item()` para esse propósito, e você também pode adicionar e remover itens de um `namedNodeMap`.

Um `namedNodeMap` é como um array, mas os itens são acessados por nome ou índice, embora este último caso seja meramente uma conveniência para enumeração, já que eles não estão em uma ordem específica na lista. O `namedNodeMap` possui um método `item()` para esse propósito, e você também pode adicionar e remover itens de um `namedNodeMap`.



Manipulando HTML com JavaScript

HTMLCollection e NodeList

HTMLCollection

Um HTMLCollection é uma coleção (lista) semelhante a um array de elementos HTML. Os elementos em uma coleção podem ser acessados por índice (começa em 0). A propriedade length retorna o número de elementos na coleção.



Manipulando HTML com JavaScript

HTMLCollection e NodeList

Quem Retorna um HTMLCollection?

O método `getElementsByTagName()`

O método `getElementsByClassName()`

A propriedade dos filhos.



Manipulando HTML com JavaScript

HTMLCollection e NodeList

Propriedades e métodos

As seguintes propriedades e métodos podem ser usados em um HTMLCollection:

Nome	Descrição
length	Retorna o número de elementos em um HTMLCollection
item()	Retorna o elemento em um índice especificado
namedItem()	Retorna o elemento com um id especificado



Manipulando HTML com JavaScript

HTMLCollection e NodeList

NodeList

Uma NodeList é uma coleção (lista) semelhante a um array de Node Objects.

Os nós em um NodeList podem ser acessados por índice (começa em 0).

A propriedade length retorna o número de nós em um NodeList.



Manipulando HTML com JavaScript

HTMLCollection e NodeList

Quem retorna um NodeList?

- O método `childNodes()`
- O método `querySelectorAll()`
- O método `getElementsByName()`



Manipulando HTML com JavaScript

HTMLCollection e NodeList

Propriedades e métodos

As seguintes propriedades e métodos podem ser usados em um NodeList:

Nome	Descrição
entry()	Retorna um Iterator com os pares chave/valor da lista
forEach()	Executa uma função de retorno de chamada para cada nó na lista
item()	Retorna o nó em um índice especificado
keys()	Retorna um Iterator com as chaves da lista
length	Retorna o número de nós em um NodeList
values()	Retorna um Iterator com os valores da lista



Manipulando HTML com JavaScript

HTMLCollection e NodeList

HTMLCollection e NodeList Não são uma matriz!

Um **HTMLCollection** e **NodeList** não são um Array!

Um **HTMLCollection** e **NodeList** pode parecer um array, mas não é.

Você pode percorrer um **HTMLCollection** e **NodeList** e fazer referência a seus elementos/nós com um índice.

Mas você não pode usar métodos Array como `push()`, `pop()` ou `join()` em um **HTMLCollection** e **NodeList** .



Manipulando HTML com JavaScript

A diferença entre HTMLCollection e NodeList

Uma NodeList e uma coleção HTML são praticamente a mesma coisa.

Ambos são coleções semelhantes a arrays (listas) de nós (elementos) extraídos de um documento. Os nós podem ser acessados por números de índice. O índice começa em 0.

Ambos possuem uma propriedade length que retorna o número de elementos da lista (coleção).

Um HTMLCollection é uma coleção de elementos de documento .

Uma NodeList é uma coleção de nós de documento (nós de elemento, nós de atributo e nós de texto).

Itens HTMLCollection podem ser acessados por seu nome, id ou número de índice.

Os itens NodeList só podem ser acessados por seu número de índice.

Uma HTMLCollection é sempre uma coleção ativa . Exemplo: Se você adicionar um elemento a uma lista no DOM, a lista no HTMLCollection também será alterada.

Um NodeList geralmente é uma coleção estática . Exemplo: Se você adicionar um elemento a uma lista no DOM, a lista em NodeList não será alterada.

Os métodos getElementByClassName() e getElementByTagName() retornam um HTMLCollection ativo.

O querySelectorAll() método retorna um NodeList estático.

A childNodes propriedade retorna uma NodeList ativa.



Manipulando HTML com JavaScript

Buscar, manipular e modificar elementos

O DOM é uma árvore, contudo ela é genérica, ou seja, pode ser atribuída a algumas categorias de documentos. Entre elas temos o documento HTML. Por isso, toda vez que precisarmos buscar, manipular, modificar, elementos na estrutura HTML começamos pelo document, depois dele acessamos outra parte da estrutura com um ‘ . ’ (ponto), no nosso exemplo, usamos um método. Que nada mais é do que “ uma função associada a um objeto, ou, simplesmente, um método é uma propriedade de um objeto que é uma função.



Manipulando HTML com JavaScript

Buscando e acessando elementos - Métodos

- `window.document.getElementById(id)`
- `window.document.getElementsByClassName(className)`
- `window.document.getElementsByTagName(tagName)`
- `window document.querySelector(cssSelector)`
- `window.document.querySelectorAll(cssSelector)`
- `window.document.element.getAttribute(nomeDoAtributo);`



Manipulando HTML com JavaScript

Alterando Elementos – Métodos e propriedades

- Propriedade `element.innerText="novo conteúdo"` :
Altera o Conteúdo de texto de um nó e seus descendentes
- Propriedade `element.textContent="novo conteúdo"` :
Altera o conteúdo textual de um nó e seus descendentes
- Propriedade `element.value="novo conteúdo"` :
Altera o valor de um elemento HTML (geralmente inputs)
- Propriedade `element.innerHTML="novo conteúdo"` :
Altera o conteúdo interno de um elemento HTML
- Propriedade `element.attribute="novo valor"` :
Altera o valor do atributo de um elemento HTML
- Propriedade `element.style.property="novo estilo"` :
Altera o estilo de um elemento HTML
- Método `element.setAttribute(attribute, value)` :
Altera o valor do atributo de um elemento HTML



Manipulando HTML com JavaScript

Adicionar e remover elementos - Métodos

- `document.createElement(element)` : Cria um elemento HTML
- `document.removeChild(element)` : Remove um elemento HTML
- `document.appendChild(element)` : Adiciona um elemento HTML
- `document.insertBefore(novo, existente)`: Insere um elemento HTML antes de um filho existente.
- `document.replaceChild(novo, anterior)` : Substitui um elemento HTML
- `document.write(text)` : Escreve no fluxo de saída HTML



Manipulando HTML com JavaScript

HTML DOMTokenList

DOMTokenList

- A DOMTokenList é um conjunto de tokens separados por espaço.
- A DOMTokenList pode ser acessado por índice (começa em 0).
- A propriedade length retorna o número de tokens em uma DOMTokenList.

Observação

A propriedade classList de um elemento HTML representa um DOMTokenList.



Manipulando HTML com JavaScript

Propriedades e métodos de DOMTokenList

Nome	Descrição
add()	Adiciona um ou mais tokens à lista
contains()	Retorna verdadeiro se a lista contiver uma classe
entry()	Retorna um Iterator com pares chave/valor da lista
forEach()	Executa uma função de retorno de chamada para cada token na lista
item()	Retorna o token em um índice especificado
keys()	Retorna um Iterator com as chaves na lista
length	Retorna o número de tokens na lista
remove()	Remove um ou mais tokens da lista
replace()	Substitui um token na lista
support()	Retorna true se um token for um dos tokens suportados de um atributo
toggle()	Alterna entre tokens na lista
value	Retorna a lista de tokens como uma string
values()	Retorna um Iterator com os valores da lista